Tema 2: Expresiones Lógicas y Operadores

Sitio: CAMPUS
Curso: Python - 2º DAW
Libro: Tema 2: Expresiones Lógicas y Operadores
Imprimido por: MARÍA HAIZHEUSKAYA
Día: jueves, 29 de octubre de 2026, 21:43

Tabla de contenidos

1. Evaluación de booleanos
2. Operadores Relacionales
3. Operadores Lógicos y Tablas de Verdad
4. Operadores de Identidad y Pertenencia
5. Guia de operadores lógicos
6. Operadores Aritméticos

1. Evaluación de booleanos

A diferencia de lenguajes como PHP o Java donde se usan minúsculas (`true` / `false`), en Python los valores booleanos siempre deben escribirse con la primera letra en mayúscula: **`True`** (Verdadero) y **`False`** (Falso).

Pertenecen al tipo de dato `bool`, que en realidad es una subclase de los números enteros (`int`). En Python, `True` equivale a 1 y `False` equivale a 0.

Además, Python tiene un concepto llamado "Truthy" y "Falsy". Cualquier objeto puede ser evaluado en un contexto booleano. Se consideran **Falsy** (evalúan a `False`):

- El propio `False`.
- El valor nulo `None`.
- El cero en cualquier tipo numérico: `0`, `0.0`.
- Secuencias o colecciones vacías: `""` (string vacío), `[]` (lista vacía), `{}` (diccionario vacío).

Cualquier otro valor se considera **Truthy** y evaluará a `True`.



2. Operadores Relacionales

Los operadores relacionales, también conocidos como operadores de comparación, se utilizan para comparar dos valores. El resultado de esta comparación siempre es un valor booleano (`True` o `False`).

Operador	Descripción	Ejemplo (x = 5, y = 10)	Resultado
`==`	Igual a (compara el valor)	`x == y`	`False`
`!=`	Distinto de	`x != y`	`True`
`<`	Menor que	`x < y`	`True`
`>`	Mayor que	`x > y`	`False`
`<=`	Menor o igual que	`x <= y`	`True`
`>=`	Mayor o igual que	`x >= y`	`False`

> **Nota para DAW:** En Python **no existe** el operador de igualdad estricta `===` como en JavaScript o PHP, ya que Python es fuertemente tipado dinámicamente y tiene otras formas de evaluar la identidad (el operador `is`).

3. Operadores Lógicos y Tablas de Verdad

Los operadores lógicos permiten combinar múltiples condiciones. A diferencia de otros lenguajes que usan `&&`, `||` y `!`, Python utiliza palabras clave en inglés puro: `and`, `or` y `not`, lo que hace el código mucho más legible.

Operador `and` (Y lógico)

Devuelve `True` solo si **todas** las condiciones son verdaderas.

Operador `or` (O lógico)

Devuelve `True` si **al menos una** de las condiciones es verdadera.

Operador `not` (Negación lógica)

Invierte el valor booleano de la expresión.

```
# Ejemplo de operadores lógicos combinados
edad = 20
tiene_carnet = True

puede_conducir = (edad >= 18) and tiene_carnet  # Resultado: True
```

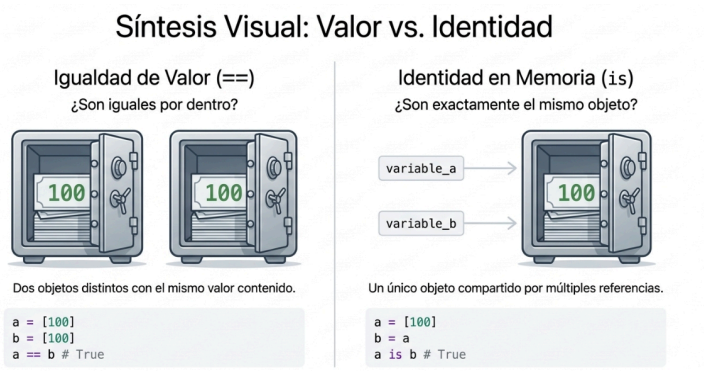
4. Operadores de Identidad y Pertenencia

Estos operadores son muy característicos de Python y resultan extremadamente útiles al trabajar con objetos, listas y cadenas de texto.

Operadores de Identidad (`is`, `is not`)

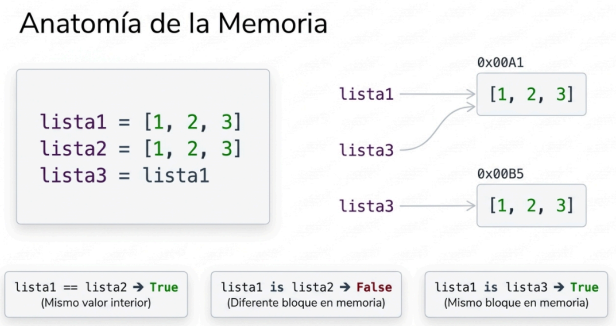
Comprueban si dos variables apuntan al **mismo objeto en la memoria**, no solo si sus valores son iguales.

- `is`: Devuelve `True` si ambas variables son exactamente el mismo objeto.
- `is not`: Devuelve `True` si las variables apuntan a objetos diferentes.




```
lista1 = [1, 2, 3]
lista2 = [1, 2, 3]
lista3 = lista1

print(lista1 == lista2)  # True (Tienen el mismo valor)
print(lista1 is lista2)  # False (Son objetos distintos en memoria)
print(lista1 is lista3)  # True (Ambas variables apuntan al mismo objeto)
```



Operadores de Pertenencia (`in`, `not in`)

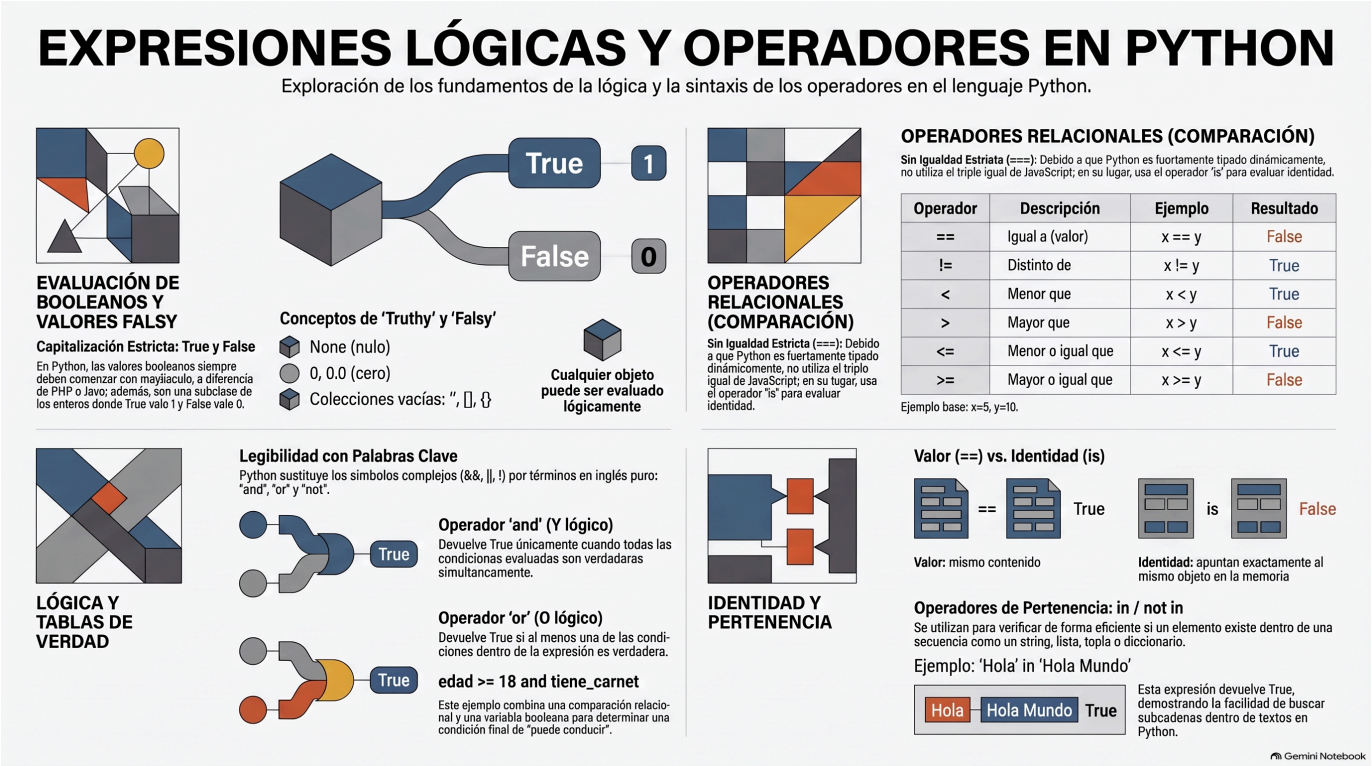
Se utilizan para comprobar si un valor o elemento se encuentra dentro de una secuencia (como un *string*, una lista, una tupla o un diccionario).

- `in`: Devuelve `True` si encuentra el valor especificado en la secuencia.
- `not in`: Devuelve `True` si NO encuentra el valor en la secuencia.

```
# Pertenencia en cadenas de texto
saludo = "Hola Mundo"
print("Hola" in saludo)  # True
print("Python" in saludo) # False

# Pertenencia en listas
lenguajes = ["Python", "JavaScript", "PHP"]
print("Java" not in lenguajes) # True
```

5. Guia de operadores lógicos



6. Operadores Aritméticos

Los operadores aritméticos en Python permiten realizar cálculos matemáticos básicos y avanzados sobre variables y valores numéricos (enteros y decimales). Son la base para cualquier algoritmo que requiera procesar datos cuantitativos.

Operadores Aritméticos Básicos

Funcionan de manera idéntica a como los usamos en las matemáticas tradicionales o en una calculadora estándar.

- **Suma (+)**: Suma dos valores. También se usa para concatenar cadenas de texto.
- **Resta (-)**: Resta el valor del lado derecho al valor del lado izquierdo.
- **Multiplicación (*)**: Multiplica dos valores.
- **División Real (/)**: Divide el primer valor entre el segundo. **Nota:** En Python 3, este operador siempre devuelve un número con decimales (*float*), incluso si la división es exacta.

Operadores Aritméticos Especiales

Python incluye operadores muy útiles para resolver problemas lógicos y matemáticos más específicos, especialmente al trabajar con números enteros.

- **División Entera (//)**: Realiza la división pero descarta por completo los decimales, devolviendo solo el número entero resultante (redondea hacia abajo).
- **Módulo (%)**: Devuelve el **resto** de una división. Es extremadamente útil para saber si un número es par o impar, o para crear ciclos periódicos.
- **Exponenciación (**)**: Eleva el número de la izquierda a la potencia indicada por el número de la derecha (base a exponente).

```
a = 10
b = 3

# Operaciones básicas
print(a + b)  # 13
print(a - b)  # 7
print(a * b)  # 30
print(a / b)  # 3.3333333333333335 (División real, devuelve un float)

# Operaciones especiales
print(a // b) # 3 (División entera, descarta el .333...)
print(a % b)  # 1 (El módulo: el resto de dividir 10 entre 3)
print(a ** b) # 1000 (Exponenciación: 10 elevado a 3)
```